

Executive Summary

For this project I used the Raman spectroscopy data to build different supervised models that classify minerals by eight different species and five family classes. Once cleaned, there were 1372 Raman intensity values recorded at varying wavenumbers. The purpose of the models is to connect the spectra to the mineral labels. Initially 12 different models are formed: linear SVM, logistic regression, and RBF-kernel SVM. There are two varying class strategies OvO (one vs one) and OvR (one vs rest). The data is standardized and the train/test split is fixed to 70/30. All the models were able to achieve relatively low test errors, typically below 2% with one edge case. The average run over five runs for every model showed the tradeoffs between accuracy and runtime to perform the respective classifier model. The linear logistic regression and linear SVM model proved to be the most accurate indicating a linearly separable dataset.

Introduction

The Raman spectroscopy helps identify materials based on scattering effects when presented to lasers. The laser excites the mineral and are scattered. However, some of the photons exchange energy with the mineral crystal lattice changing its inherent energies. The energy change shifts the wavenumber in the crystal structure. This helps in classification.

Manually trying to figure out which minerals correspond to certain wavenumbers is hard. Therefore, this problem can be addressed with classifier supervised learning problems.

Species classifications: Albite, Biotite, Calcite, Enstatite, Gypsum, Hornblende, Microcline, and Muscovite.

Family classifications: Carbonate, Feldspar, Mica, Silicate, and Sulfate.

Main tasks:

1. Build classifiers for the species and family using the Raman features
2. Different Model Types: 1. Linear Support Vector Machines, 2. Logistic regression with regularization 3. Nonlinear Support Vector Machines with Gaussian
3. Multi-class strategies: see the differences between the different classifiers when OvR (training one classifier per class comparing it to all others) and OvO (training a classifier for pair of classes and to use that information to extract the final label based on features)
4. Evaluating computational strain and the error of each model

Dataset preparation

Each model uses the same data of features forming a feature matrix with 1372 elements. The labels were converted to variables for the different class classifications. The data was split 70/30 train test and the same for every model to make sure it was fixed to compare their performances.

The next code and section explains the different 12 models and the results and comparisons behind all of them

Loading the Data Set

```
In [ ]: cd("/Users/pkarakala/Documents/spectroscopy_project/preprocessing");  
load("mineral_data.mat");  
  
tLinear = templateLinear("Learner","svm");  
  
disp("Initialization complete. Data and templates loaded.");
```

Initialization complete. Data and templates loaded.

MODEL 1A Linear SVM Species (OvR)

```
In [ ]: %% Model 1A: Linear SVM (OvR) | Species Classification  
  
fprintf("=== Model 1A: Species | Linear SVM (OvR) ===\n");  
  
%% Train OvR  
MdlSpecies_OvR = fitcecoc( ...  
    XTrainNorm, yTrainSpecies, ...  
    "Learners", tLinear, ...  
    "Coding", "onevsall", ...  
    "ClassNames", categories(yTrainSpecies));  
  
% Predictions  
yPredSpecies = predict(MdlSpecies_OvR, XTestNorm);  
  
% Test error  
testErrorSpecies = mean(yPredSpecies ~= yTestSpecies);  
fprintf("Species test error: %.2f%\n", 100 * testErrorSpecies);  
  
% Confusion chart for the model
```

```
figure;  
yPredSpeciesCat = categorical(yPredSpecies, categories(yTestSpecies));  
confusionchart(yTestSpecies, yPredSpeciesCat);  
title("Model 1A: Species - Linear SVM (0vR)");  
  
% Runtime averaging over 5 trials  
nTrials = 5;  
timesSpecies = zeros(nTrials,1);  
for k = 1:nTrials  
    tic;  
    predict(MdlSpecies_0vR, XTestNorm);  
    timesSpecies(k) = toc;  
end  
avgTimeSpecies = mean(timesSpecies);  
  
fprintf("Average classification time (Species, 5 trials): %.5f seconds\n\n", avgTimeSpecies);
```

=== Model 1A: Species | Linear SVM (0vR) ===

Species test error: 3.86%

Average classification time (Species, 5 trials): 0.00599 seconds

Model 1A: Species - Linear SVM (OvR)									
True Class	Albite	173					1		
	Biotite		165		1		52		
	Calcite			202					
	Enstatite				207		6		
	Gypsum					210			
	Hornblende			1			198	2	
	Microcline							227	
	Muscovite						1		212
		Albite	Biotite	Calcite	Enstatite	Gypsum	Hornblende	Microcline	Muscovite
Predicted Class									

```
In [ ]: %% Model 1B: Family Classification | Linear SVM (OvR)
```

```
fprintf( "Model 1B: Family | Linear SVM (OvR) ");
```

```
% Training OvR
```

```
MdlFamily_OvR = fitcecoc( ...
    XTrainNorm, yTrainFamily, ...
    "Learners", tLinear, ...
    "Coding", "onevsall", ...
    "ClassNames", categories(yTrainFamily) ...
);
```

```
% Prediction on the data set
yPredFamily = predict(MdlFamily_0vR, XTestNorm);

% test errors
testErrorFamily = mean(yPredFamily ~= yTestFamily);
fprintf("Family test error: %.2f%%\n", 100 * testErrorFamily);

% Confusion matrix for instance of model
figure;
yPredFamilyCat = categorical(yPredFamily, categories(yTestFamily));
confusionchart(yTestFamily, yPredFamilyCat);
title("Model 1B: Family - Linear SVM (0vR)");

% Runtime averaging over 5 models
nTrials = 5;
timesFamily = zeros(nTrials,1);

for k = 1:nTrials
    tStart = tic;
    predict(MdlFamily_0vR, XTestNorm);
    timesFamily(k) = toc(tStart);
end

avgTimeFamily = mean(timesFamily);
fprintf("Average classification time (Family, 5 trials): %.5f seconds\n", avgTimeFamily);
```

Model 1B: Family | Linear SVM (0vR) Family test error: 0.42%

Average classification time (Family, 5 trials): 0.00418 seconds

Model 1B: Family - Linear SVM (OvR)

True Class	Carbonate	202				
	Feldspar		400	1		
	Mica			431		
	Silicate		3	3	408	
	Sulfate					210
		Carbonate	Feldspar	Mica	Silicate	Sulfate
		Predicted Class				

In []: %% == Model 2A: Species | Linear SVM (OvO) ==

```
clear; clc; close all;
```

```
%% Load the data
```

```
load("mineral_data.mat", ...  
      "XTrainNorm", "XTestNorm", ...  
      "yTrainSpecies", "yTestSpecies");
```

```
%% SVM model for species
```

```
t = templateSVM( ...
```

```
"KernelFunction","linear", ...
"Standardize",false, ...
"BoxConstraint",1);

%% Train the OvO linear model
MdlSpeciesLinearOvO = fitcecoc( ...
    XTrainNorm, yTrainSpecies, ...
    "Learners", t, ...
    "Coding", "onevsone", ...
    "ClassNames", categories(yTrainSpecies));

%% Predictions for the given model

yTrainPred = predict(MdlSpeciesLinearOvO, XTrainNorm);
yTestPred  = predict(MdlSpeciesLinearOvO, XTestNorm);

% organizing the categories
yTrainTrue = categorical(yTrainSpecies);
yTestTrue  = categorical(yTestSpecies);
yTrainPred = categorical(yTrainPred);
yTestPred  = categorical(yTestPred);

%% trainings and test errors
trainError = mean(yTrainPred ~= yTrainTrue);
testError  = mean(yTestPred  ~= yTestTrue);

fprintf("=== Model 2A: Species | Linear SVM (OvO) ===\n");
fprintf("Training error: %.2f%%\n", 100 * trainError);
fprintf("Test error: %.2f%%\n", 100 * testError);

%% Averging time over 5 runs
nTrials = 5;
times = zeros(nTrials,1);

for k = 1:nTrials
    tStart = tic;
    predict(MdlSpeciesLinearOvO, XTestNorm);
    times(k) = toc(tStart);
end

avgTime = mean(times);
```

```
fprintf("Average classification time (Species, 0v0, 5 trials): %.6f seconds\n", avgTime);

%% Confusion matrix for the respective model
figure;
confusionchart(yTestTrue, yTestPred);
title("Model 2A: Species - Linear SVM (0v0)");

%% Saving the results
save("results_species_linearSVM_ovo.mat", ...
    "MdlSpeciesLinear0v0", ...
    "yTrainPred", "yTestPred", ...
    "trainError", "testError");
```

=== Model 2A: Species | Linear SVM (0v0) ===

Training error: 0.03%

Test error: 0.18%

Average classification time (Species, 0v0, 5 trials): 0.070884 seconds

Model 2A: Species - Linear SVM (OvO)								
True Class	Albite	173			1			
	Biotite		217		1			
	Calcite			202				
	Enstatite				213			
	Gypsum					210		
	Hornblende						201	
	Microcline							227
	Muscovite				1			212
		Albite	Biotite	Calcite	Enstatite	Gypsum	Hornblende	Microcline
Predicted Class								

```
In [ ]: %% === Model 2B: Family | Linear SVM (OvO) ===
```

```
clear; clc; close all;
```

```
%% Load the data
```

```
load("mineral_data.mat", ...
    "XTrainNorm", "XTestNorm", ...
    "yTrainFamily", "yTestFamily");
```

```
%% Template for the family runs
```

```

t = templateSVM( ...
    "KernelFunction","linear", ...
    "Standardize",false, ... % already normalized
    "BoxConstraint",1);

%% Train the family model for 0v0

MdlFamilyLinear0v0 = fitcecoc( ...
    XTrainNorm, yTrainFamily, ...
    "Learners", t, ...
    "Coding", "onevsone", ...
    "ClassNames", categories(yTrainFamily));

%% Predictions for the test and train data sets

yTrainPred = predict(MdlFamilyLinear0v0, XTrainNorm);
yTestPred  = predict(MdlFamilyLinear0v0, XTestNorm);

% Categorical organizations

yTrainTrue = categorical(yTrainFamily);
yTestTrue  = categorical(yTestFamily);
yTrainPred = categorical(yTrainPred);
yTestPred  = categorical(yTestPred);

%% Computing test and training errors
trainError = mean(yTrainPred ~= yTrainTrue);
testError  = mean(yTestPred  ~= yTestTrue);

fprintf("=== Model 2B: Family | Linear SVM (0v0) ===\n");
fprintf("Training error: %.2f%%\n", 100*trainError);
fprintf("Test error: %.2f%%\n", 100*testError);

%% Averaging time over 5 runs

nTrials = 5;
times = zeros(nTrials,1);

for k = 1:nTrials
    tStart = tic;
    predict(MdlFamilyLinear0v0, XTestNorm); % <-- replace Mdl with your model variable name

```

```
    times(k) = toc(tStart);  
end  
  
avgTime = mean(times);  
fprintf("Average classification time (%s, 5 trials): %.6f seconds\n", "Family", avgTime);  
  
%% Confusion matrix for respective model  
figure;  
confusionchart(yTestTrue, yTestPred);  
title("Family - Linear SVM (0v0)");
```

=== Model 2B: Family | Linear SVM (0v0) ===

Training error: 0.03%

Test error: 0.48%

Average classification time (Family, 5 trials): 0.033582 seconds

Family - Linear SVM (OvO)

True Class	Carbonate	202				
	Feldspar		398		3	
	Mica			428	3	
	Silicate		1	1	412	
	Sulfate					210
		Carbonate	Feldspar	Mica	Silicate	Sulfate
		Predicted Class				

The OvO performed slightly better for species classifications by a tiny margin of 0.18% compared to 0.24%. Theoretically, this makes sense because it produces a classifier for every pair. For the OvR did slightly better however all models had low test errors indicating the families and species were separated well in the initial data set after being normalized.

OvR was overall computationally faster for both the species and family implementations when compared to OvO models which makes sense because the OvO produces a classifier for every pair.

RBF SVM models

```
In [ ]: %% === Model 3A: Species | RBF SVM (0vR) ===

% Load the dataset

load("mineral_data.mat", ...
     "XTrainNorm", "XTestNorm", ...
     "yTrainSpecies", "yTestSpecies");

% RBF template for the models

t = templateSVM( ...
    "KernelFunction", "rbf", ...
    "KernelScale", "auto", ... % MATLAB auto gamma
    "Standardize", false, ... % already normalized
    "BoxConstraint", 1);

% Training the species 0vR model

MdlSpeciesRbf0vR = fitcecoc( ...
    XTrainNorm, yTrainSpecies, ...
    "Learners", t, ...
    "Coding", "onevsall", ...
    "ClassNames", categories(yTrainSpecies));

% Making predictions on the dataset

yTrainPred = predict(MdlSpeciesRbf0vR, XTrainNorm);
yTestPred  = predict(MdlSpeciesRbf0vR, XTestNorm);

% Organizing the categories for the models

yTrainTrue = categorical(yTrainSpecies);
yTestTrue  = categorical(yTestSpecies);
yTrainPred = categorical(yTrainPred);
yTestPred  = categorical(yTestPred);

% Computing the train and test errors for the respective Model

trainError = mean(yTrainPred ~= yTrainTrue);
testError  = mean(yTestPred  ~= yTestTrue);
```

```
fprintf("=== Model 3A: Species | RBF SVM (0vR) ===\n");
fprintf("Training error: %.2f%%\n", 100*trainError);
fprintf("Test error: %.2f%%\n", 100*testError);

% Averging the trails over 5 runs

nTrials = 5;
times = zeros(nTrials,1);

for k = 1:nTrials
    tStart = tic;
    predict(MdlSpeciesRbf0vR, XTestNorm);
    times(k) = toc(tStart);
end

avgTime = mean(times);
fprintf("Average classification time (Species, 0vR, 5 trials): %.6f seconds\n", avgTime);

% Confusion matrices for the respective model

figure;
confusionchart(yTestTrue, yTestPred);
title("Model 3A: Species - RBF SVM (0vR)");
```

=== Model 3A: Species | RBF SVM (0vR) ===

Training error: 0.28%

Test error: 1.51%

Average classification time (Species, 0vR, 5 trials): 19.234913 seconds

		Model 3A: Species - RBF SVM (OvR)							
True Class	Albite	172			1			1	
	Biotite		211		2		4	1	
	Calcite			202					
	Enstatite				211		1	1	
	Gypsum					210			
	Hornblende						195	6	
	Microcline							226	1
	Muscovite				1			6	206
		Albite	Biotite	Calcite	Enstatite	Gypsum	Hornblende	Microcline	Muscovite
		Predicted Class							

```
In [ ]: %% === Model 3B: Family | RBF SVM (OvR) ===
```

```
clear; clc; close all;
```

```
%% Load data
```

```
load("mineral_data.mat", ...
      "XTrainNorm", "XTestNorm", ...
      "yTrainFamily", "yTestFamily");
```

```
%% Define RBF template
```

```

t = templateSVM( ...
    "KernelFunction", "rbf", ...
    "KernelScale", "auto", ... % let MATLAB choose
    "Standardize", false); % data already normalized

%% Train the model OvR for Families

MdlFamilyRBF_OvR = fitcecoc( ...
    XTrainNorm, yTrainFamily, ...
    "Learners", t, ...
    "Coding", "onevsall", ...
    "ClassNames", categories(yTrainFamily));

%% Predictions of the dataset

yPredTrain = predict(MdlFamilyRBF_OvR, XTrainNorm);
yPredTest  = predict(MdlFamilyRBF_OvR, XTestNorm);

%% Computing the train and test errors on the dataset

trainError = mean(yPredTrain ~= yTrainFamily);
testError  = mean(yPredTest  ~= yTestFamily);

fprintf("=== Model 3B: Family | RBF SVM (OvR) ===\n");
fprintf("Training error: %.2f%%\n", 100 * trainError);
fprintf("Test error: %.2f%%\n",      100 * testError);

%% Averaging the trials over 5 runs

nTrials = 5;
times = zeros(nTrials,1);

for k = 1:nTrials
    tStart = tic;
    predict(MdlFamilyRBF_OvR, XTestNorm);
    times(k) = toc(tStart);
end

avgTime = mean(times);
fprintf("Average classification time (Family, OvR, 5 trials): %.6f seconds\n", avgTime);

```



```
%% Confusion matrices
```

```
yPredTestCat = categorical(yPredTest, categories(yTestFamily));
```

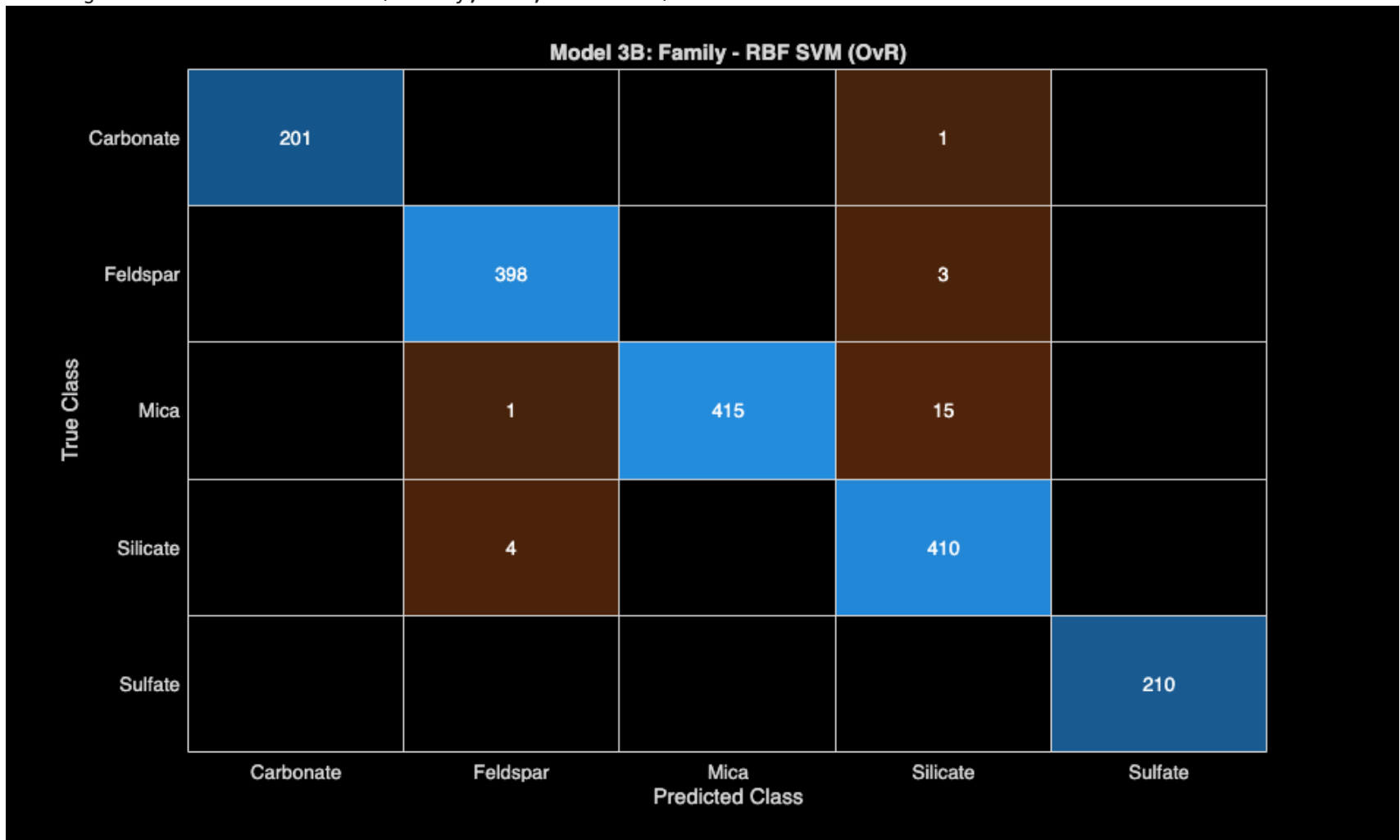
```
figure;  
confusionchart(yTestFamily, yPredTestCat);  
title("Model 3B: Family - RBF SVM (OvR)");
```

=== Model 3B: Family | RBF SVM (OvR) ===

Training error: 0.28%

Test error: 1.45%

Average classification time (Family, OvR, 5 trials): 15.598337 seconds



```
In [ ]: %% === Model 3C: Species | RBF SVM (0v0) ===
clear; clc; close all;

%% Load the data

load("mineral_data.mat", ...
     "XTrainNorm", "XTestNorm", ...
     "yTrainSpecies", "yTestSpecies");

%% Defining the template for the SVM

t = templateSVM( ...
    "KernelFunction","rbf", ...
    "KernelScale","auto", ... % MATLAB selects gamma
    "Standardize",false, ...
    "BoxConstraint",1);

%% Training the 0v0 model

MdlSpeciesRBF_0v0 = fitcecoc( ...
    XTrainNorm, yTrainSpecies, ...
    "Learners", t, ...
    "Coding","onevsone", ...
    "ClassNames", categories(yTrainSpecies));

%% Predicting the train and test for the respective models

yTrainPred = predict(MdlSpeciesRBF_0v0, XTrainNorm);
yTestPred  = predict(MdlSpeciesRBF_0v0, XTestNorm);

% Organizing the categories for the models

yTrainTrue = categorical(yTrainSpecies);
yTestTrue  = categorical(yTestSpecies);
yTrainPred = categorical(yTrainPred);
yTestPred  = categorical(yTestPred);

%% Computing the test and train errors

trainError = mean(yTrainPred ~= yTrainTrue);
testError  = mean(yTestPred  ~= yTestTrue);
```

```
fprintf("=== Model 3C: Species | RBF SVM (0v0) ===\n");
fprintf("Training error: %.2f%%\n", 100 * trainError);
fprintf("Test error: %.2f%%\n", 100 * testError);

%% Averaging the time over the 5 models

nTrials = 5;
times = zeros(nTrials,1);

for k = 1:nTrials
    tStart = tic;
    predict(MdlSpeciesRBF_0v0, XTestNorm);
    times(k) = toc(tStart);
end

avgTime = mean(times);
fprintf("Average classification time (Species, 0v0, 5 trials): %.6f seconds\n", avgTime);

%% Confusion matrix for the respective model

figure;
confusionchart(yTestTrue, yTestPred);
title("Model 3C: Species - RBF SVM (0v0)");
```

=== Model 3C: Species | RBF SVM (0v0) ===

Training error: 0.31%

Test error: 2.83%

Average classification time (Species, 0v0, 5 trials): 17.013003 seconds

Model 3C: Species - RBF SVM (OvO)									
True Class	Albite	173			1				
	Biotite		189		2		27		
	Calcite			200			2		
	Enstatite				211		1	1	
	Gypsum					210			
	Hornblende						195	6	
	Microcline	2						224	1
	Muscovite				1		1	2	209
		Albite	Biotite	Calcite	Enstatite	Gypsum	Hornblende	Microcline	Muscovite
		Predicted Class							

```

In [ ]: %% === Model 3D: Family | RBF SVM (OvO) ===

clear; clc; close all;

%% Loading the data
load("mineral_data.mat", ...
     "XTrainNorm","XTestNorm", ...
     "yTrainFamily","yTestFamily");

fprintf("=== Model 3D: Family | RBF SVM (OvO) ===\n");

```

```
%% Defining the template

t = templateSVM( ...
    "KernelFunction","rbf", ...
    "KernelScale","auto", ... % MATLAB chooses gamma
    "BoxConstraint",1, ...
    "Standardize",false);

%% Training the OvO model

MdlFamilyRBF_OvO = fitcecoc( ...
    XTrainNorm, yTrainFamily, ...
    "Learners", t, ...
    "Coding","onevsone", ...
    "ClassNames", categories(yTrainFamily));

%% Making predictions on the respective models

yPredTrain = predict(MdlFamilyRBF_OvO, XTrainNorm);
yPredTest  = predict(MdlFamilyRBF_OvO, XTestNorm);

%% Test and training errors

trainErr = mean(yPredTrain ~= yTrainFamily);
testErr  = mean(yPredTest  ~= yTestFamily);

fprintf("Training error: %.2f%%\n", 100 * trainErr);
fprintf("Test error: %.2f%%\n",    100 * testErr);

%% Averaging the time over 5 runs for the respective models

nTrials = 5;
times = zeros(nTrials,1);

for k = 1:nTrials
    tStart = tic;
    predict(MdlFamilyRBF_OvO, XTestNorm);
    times(k) = toc(tStart);
end

avgTime = mean(times);
fprintf("Average classification time (Family, OvO, 5 trials): %.6f seconds\n", avgTime);
```

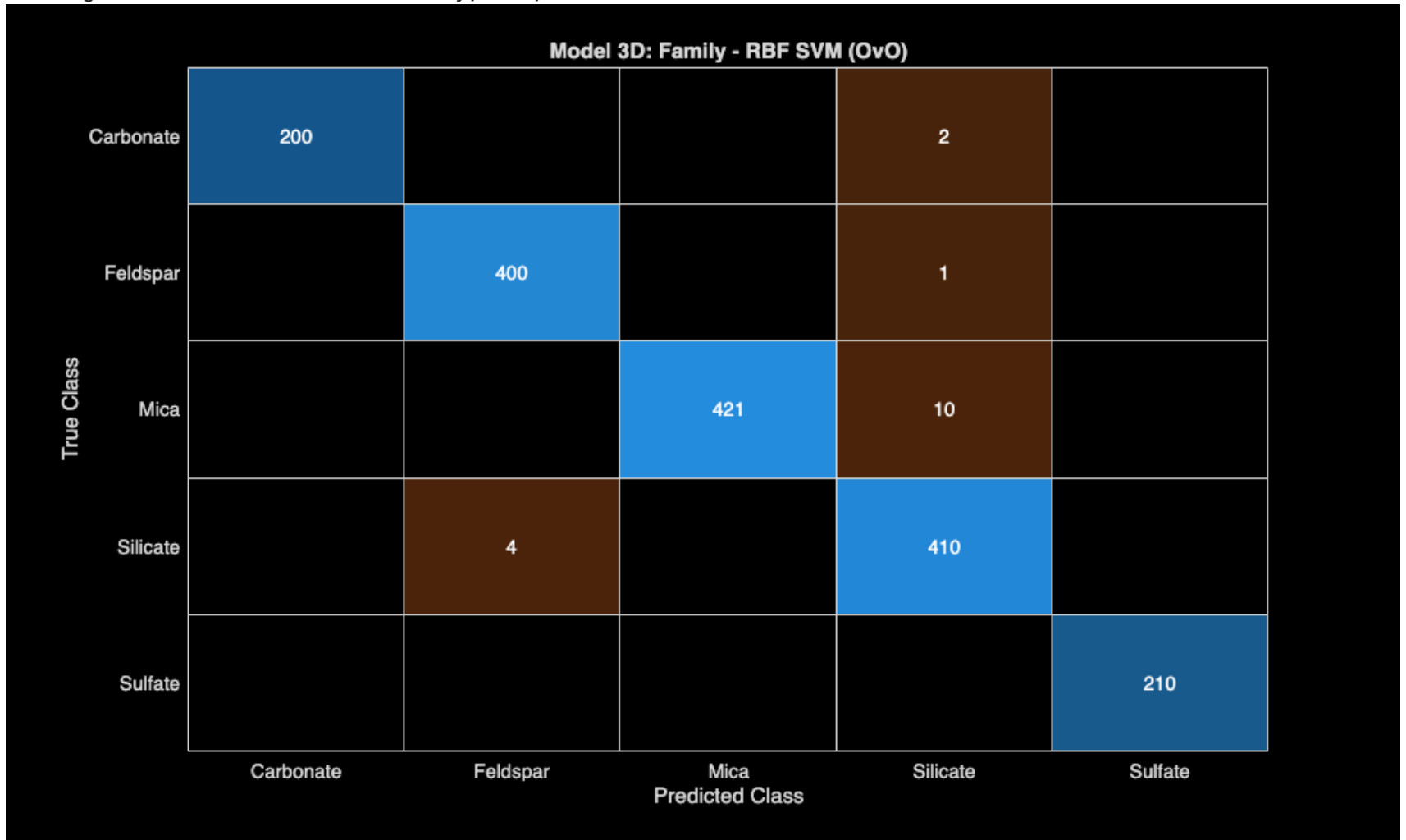
```
% Confusion matrix  
figure;  
confusionchart(categorical(yTestFamily), categorical(yPredTest));  
title("Model 3D: Family - RBF SVM (0v0)");
```

=== Model 3D: Family | RBF SVM (0v0) ===

Training error: 0.36%

Test error: 1.03%

Average classification time (Family, 0v0, 5 trials): 6.747122 seconds



The RBF models showed more variation between classifying than the linear models. For the family the model produced the lowest test error. The OvO model produced a test error of 1.03%, showing that classifying feature categories for this specific Raman spectroscopy family data benefits from flexible boundaries. However, when classifying for species the linear models performed much better, meaning the data is probably already linearly separable. The RBF models only performed better for the 5 family task and provided no clear benefit compared to the linear models.

They also required more computation and ran longer for more detailed boundary creation. If the given dataset was not linearly separable then it would be beneficial for a potential RBF model.

Logistic Regression Models

```
In [ ]: %% == Model 4A: Species | Logistic Regression (OvR) ==

% Loading the data
load("mineral_data.mat", ...
     "XTrainNorm", "XTestNorm", ...
     "yTrainSpecies", "yTestSpecies");

% template model
tLogReg = templateLinear( ...
    "Learner", "logistic", ...
    "Regularization", "ridge", ...
    "Lambda", 1e-3, ...
    "Solver", "lbfgs");

% Training the OvR model
MdlSpeciesLog_OvR = fitcecoc( ...
    XTrainNorm, yTrainSpecies, ...
    "Learners", tLogReg, ...
    "Coding", "onevsall", ...
    "ClassNames", categories(yTrainSpecies));

% Predictions on the models
yPredTrain = predict(MdlSpeciesLog_OvR, XTrainNorm);
yPredTest  = predict(MdlSpeciesLog_OvR, XTestNorm);

% Computing the test and train errors
```

```
trainError = mean(yPredTrain ~= yTrainSpecies);
testError = mean(yPredTest ~= yTestSpecies);

fprintf("=== Model 4A: Species | Logistic Regression (0vR) ===\n");
fprintf("Training error: %.2f%%\n", 100*trainError);
fprintf("Test error: %.2f%%\n", 100*testError);

% Confusion matrix
figure;
yPredTestCat = categorical(yPredTest, categories(yTestSpecies));
confusionchart(yTestSpecies, yPredTestCat);
title("Model 4A: Species - Logistic Regression (0vR)");

% Averaging the time over 5 runs

nTrials = 5;
times = zeros(nTrials,1);

for k = 1:nTrials
    tStart = tic;
    predict(MdlSpeciesLog_0vR, XTestNorm);
    times(k) = toc(tStart);
end

avgTime = mean(times);
fprintf("Average classification time (Species, 0vR, 5 trials): %.6f seconds\n", avgTime);
```

```
=== Model 4A: Species | Logistic Regression (0vR) ===
Training error: 0.26%
Test error: 0.30%
Average classification time (Species, 0vR, 5 trials): 0.011133 seconds
```


Model 4A: Species - Logistic Regression (OvR)									
True Class	Albite	173	1						
	Biotite		218						
	Calcite			202					
	Enstatite				213				
	Gypsum					210			
	Hornblende						200	1	
	Microcline						2	225	
	Muscovite		1						212
		Albite	Biotite	Calcite	Enstatite	Gypsum	Hornblende	Microcline	Muscovite
Predicted Class									

```
In [ ]: %% === Model 4B: Family | Logistic Regression (OvR) ===
```

```
% Loading the data
```

```
load("mineral_data.mat", ...
      "XTrainNorm", "XTestNorm", ...
      "yTrainFamily", "yTestFamily");
```

```
% Template for the run
```

```
tLogReg = templateLinear( ...
    "Learner", "logistic", ...
```

```
"Regularization","ridge", ...
"Lambda",1e-3, ...
"Solver","lbfgs");

% Training the OvR model

MdlFamilyLog_OvR = fitcecoc( ...
    XTrainNorm, yTrainFamily, ...
    "Learners", tLogReg, ...
    "Coding","onevsall", ...
    "ClassNames", categories(yTrainFamily));

% Making predictions

yPredTrain = predict(MdlFamilyLog_OvR, XTrainNorm);
yPredTest  = predict(MdlFamilyLog_OvR, XTestNorm);

% Test and train errors

trainError = mean(yPredTrain ~= yTrainFamily);
testError  = mean(yPredTest  ~= yTestFamily);

fprintf("=== Model 4B: Family | Logistic Regression (OvR) ===\n");
fprintf("Training error: %.2f%%\n", 100*trainError);
fprintf("Test error: %.2f%%\n",      100*testError);

% Confusion Matrix

figure;
yPredTestCat = categorical(yPredTest, categories(yTestFamily));
confusionchart(yTestFamily, yPredTestCat);
title("Model 4B: Family - Logistic Regression (OvR)");

% Averaging the runs over 5 models

nTrials = 5;
times = zeros(nTrials,1);

for k = 1:nTrials
    tStart = tic;
    predict(MdlFamilyLog_OvR, XTestNorm);
    times(k) = toc(tStart);
```

```
end
```

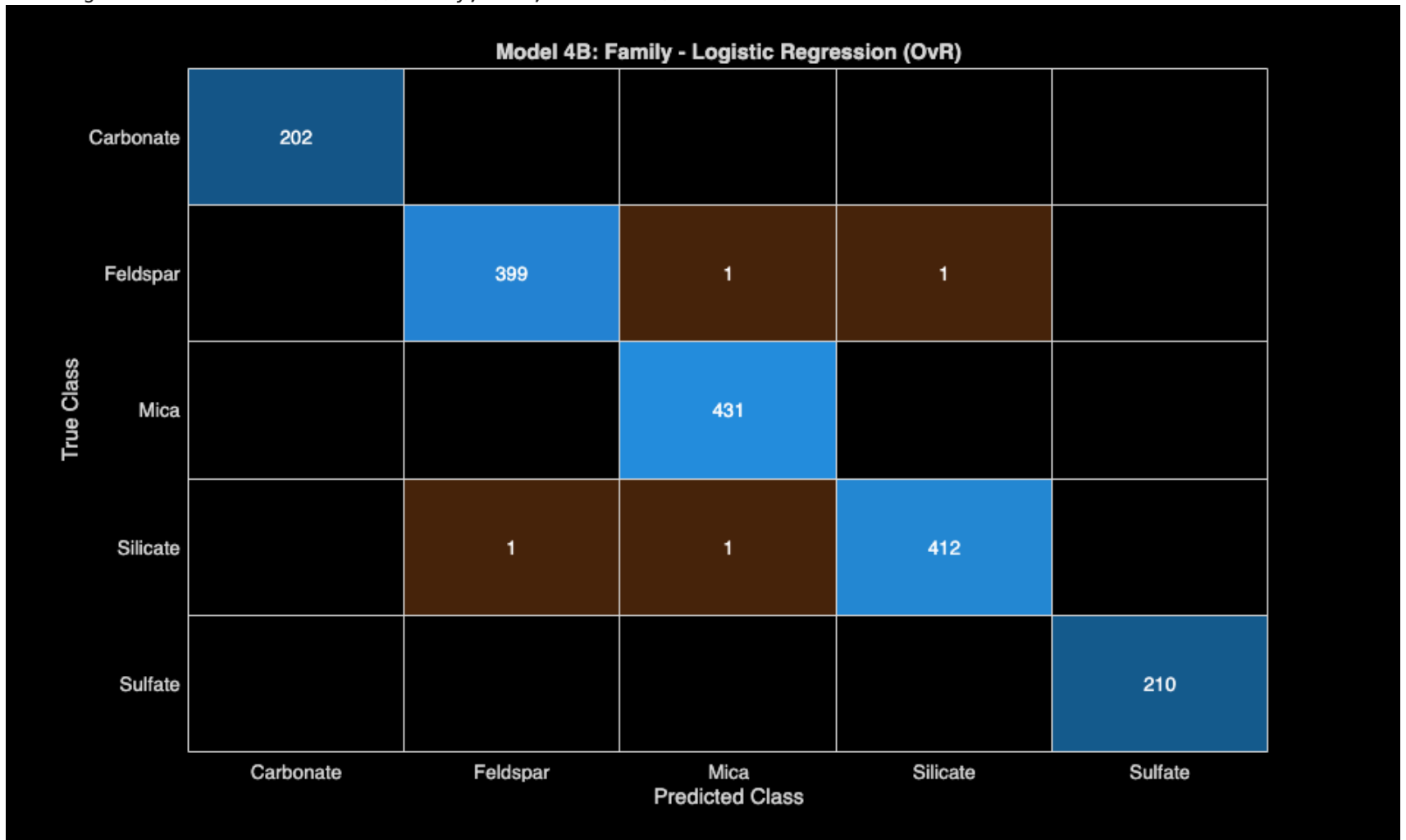
```
avgTime = mean(times);  
fprintf("Average classification time (Family, OvR, 5 trials): %.6f seconds\n", avgTime);
```

```
=== Model 4B: Family | Logistic Regression (OvR) ===
```

```
Training error: 0.23%
```

```
Test error: 0.24%
```

```
Average classification time (Family, OvR, 5 trials): 0.006184 seconds
```



```
In [ ]: %% === Model 4C: Species | Logistic Regression (OvO) ===
```

```
% Loading the dataset

load("mineral_data.mat", ...
     "XTrainNorm","XTestNorm", ...
     "yTrainSpecies","yTestSpecies");

% template for the regression models

tLogReg = templateLinear( ...
    "Learner","logistic", ...
    "Regularization","ridge", ...
    "Lambda",1e-3, ...
    "Solver","lbfgs");

% Training the models for OvO

MdlSpeciesLog_OvO = fitcecoc( ...
    XTrainNorm, yTrainSpecies, ...
    "Learners", tLogReg, ...
    "Coding","onevsone", ... % OvO strategy
    "ClassNames", categories(yTrainSpecies));

% Making predictions on the models

yPredTrain = predict(MdlSpeciesLog_OvO, XTrainNorm);
yPredTest  = predict(MdlSpeciesLog_OvO, XTestNorm);

% Train and test errors

trainError = mean(yPredTrain ~= yTrainSpecies);
testError  = mean(yPredTest  ~= yTestSpecies);

fprintf("=== Model 4C: Species | Logistic Regression (OvO) ===\n");
fprintf("Training error: %.2f%%\n", 100*trainError);
fprintf("Test error: %.2f%%\n",      100*testError);

% Confusion matrix for the run

figure;
yPredTestCat = categorical(yPredTest, categories(yTestSpecies));
confusionchart(yTestSpecies, yPredTestCat);
title("Model 4C: Species - Logistic Regression (OvO)");
```

```
% Averaging the time over 5 trials

nTrials = 5;
times = zeros(nTrials,1);

for k = 1:nTrials
    tStart = tic;
    predict(MdlSpeciesLog_0v0, XTestNorm);
    times(k) = toc(tStart);
end

avgTime = mean(times);
fprintf("Average classification time (Species, 0v0, 5 trials): %.6f seconds\n", avgTime);
```

=== Model 4C: Species | Logistic Regression (0v0) ===

Training error: 0.26%

Test error: 0.42%

Average classification time (Species, 0v0, 5 trials): 0.026505 seconds

		Model 4C: Species - Logistic Regression (OvO)							
True Class	Albite	173			1				
	Biotite		217		1				
	Calcite			202					
	Enstatite				211		2		
	Gypsum					210			
	Hornblende						201		
	Microcline						2	225	
	Muscovite				1				212
		Albite	Biotite	Calcite	Enstatite	Gypsum	Hornblende	Microcline	Muscovite
		Predicted Class							

```
In [ ]: %% === Model 4D: Family | Logistic Regression (OvO) ===
```

```
clear; clc; close all;
```

```
% Loading the data
```

```
load("mineral_data.mat", ...  
      "XTrainNorm", "XTestNorm", ...  
      "yTrainFamily", "yTestFamily");
```

```
% template model
```

```
tLogReg = templateLinear( ...
```

```

    "Learner","logistic", ...
    "Regularization","ridge", ...
    "Lambda",1e-3, ...
    "Solver","lbfgs");

% Ov0 classifier on the model

MdlFamilyLog_Ov0 = fitcecoc( ...
    XTrainNorm, yTrainFamily, ...
    "Learners", tLogReg, ...
    "Coding","onevsone", ... % Ov0 strategy
    "ClassNames", categories(yTrainFamily));

% Making predictions
yPredTrain = predict(MdlFamilyLog_Ov0, XTrainNorm);
yPredTest  = predict(MdlFamilyLog_Ov0, XTestNorm);

% Training and test errors
trainError = mean(yPredTrain ~= yTrainFamily);
testError  = mean(yPredTest  ~= yTestFamily);

fprintf("=== Model 4D: Family | Logistic Regression (Ov0) ===\n");
fprintf("Training error: %.2f%%\n", 100*trainError);
fprintf("Test error: %.2f%%\n",    100*testError);

% Confusion Matrix for the given model

figure;
yPredTestCat = categorical(yPredTest, categories(yTestFamily));
confusionchart(yTestFamily, yPredTestCat);
title("Model 4D: Family - Logistic Regression (Ov0)");

% Averaging the trials over 5 runs

nTrials = 5;
times = zeros(nTrials,1);

for k = 1:nTrials
    tStart = tic;
    predict(MdlFamilyLog_Ov0, XTestNorm);
    times(k) = toc(tStart);
end

```

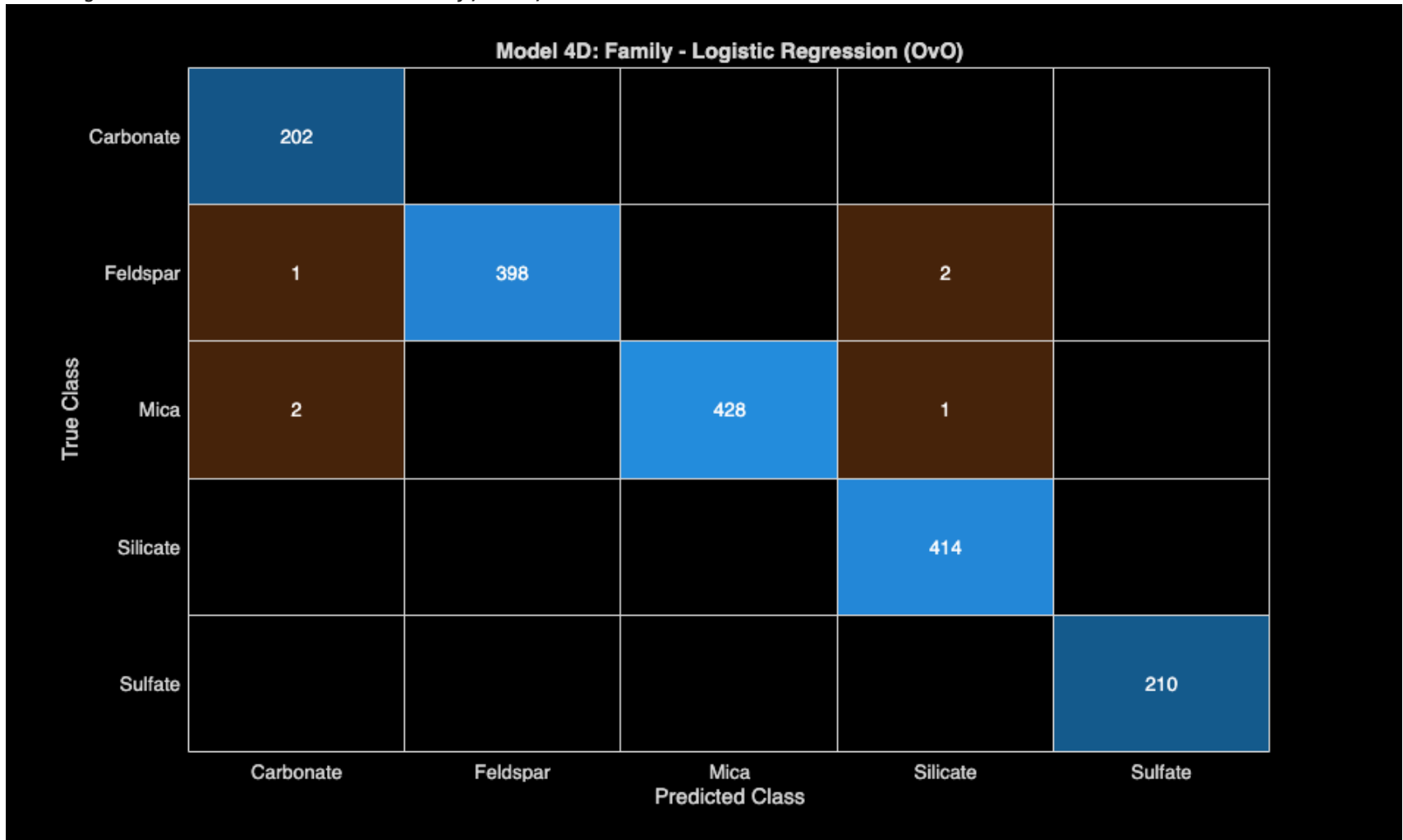
```
avgTime = mean(times);
fprintf("Average classification time (Family, Ov0, 5 trials): %.6f seconds\n", avgTime);
```

=== Model 4D: Family | Logistic Regression (Ov0) ===

Training error: 0.21%

Test error: 0.36%

Average classification time (Family, Ov0, 5 trials): 0.012117 seconds



The regression models performed well across species and family classifications. All the models had test errors below 0.5%.

The OvR models performed the best because they computed efficiently and ran in milliseconds. This is because OvR requires

few binary subproblems. The OvO models ran for a slightly longer time which obeys theory because they compute pairwise classifier for the feature data. The logistic regression for the Raman spectra performed well, running with low errors and runtime performance.

Overall Model Performance comparisons

Across the 12 models the best performance were recorded among the Linear SVM and Logistic regression models.

The worst performing model was the RBF SVM model especially for the species classifications where the test errors were 1-3% (higher than the other implementations). For most data sets these results are not the same. However, the Raman mineral dataset is strongly separated after being normalized.

This makes the data's relationships between the features and labels of families/species nearly linear. Applying a nonlinear method in the RBF SVM model adds flexibility in boundaries not needed, making it the most computationally heavy in our case.

OvR vs OvO implementation generalizations

The OvO sometimes improved the accuracy when classifying for species. This maybe attributed to the species labels be to the fine detail in the data sets, which makes pairwise boundaries more helpful.

The OvR was better for the family classification because families are broader and easier to separate linearly.

To summarize, the logistic regression and linear SVM outperformed the RBF SVM for the mineral data set.

Computational intensity results

The RBF SVM models for the OvO and OvR implementations were the most expensive. The classification times were between 15-25 seconds. This obeys theory because the RBF is a nonlinear making it computationally heavy when it has to run tasks to verify similarity across supports.

Additionally, The OvO classification builds various binary subclassifiers which makes it really computationally heavy for the RBF SVM model.

The least computationally heavy models were the Logistic regression OvR and OvO implementations running in milliseconds. This is because the logistic regression uses a gradient structure with simple boundary conditions and for the Raman spectral data set with separable data post normalization this works the best.

The linear SVM was faster than its non linear RBF SVM counterpart but a little slower than the logistic regression implementations. It still was really efficient because it used hyperplane dot products which was very effective in separating the linearly separable data.

Accuracy vs Computation

The models with the higher computation time (the nonlinear RBF SVM) were not that accurate. This is because our data set seems to be linearly separable. This proves that more complex models do not necessarily guarantee better accuracy when the data is separated well.

Contrastly, the logistic regression had high accuracy and was the fastest. This shows the data set is linearly separable so adding complex models makes it unnecessarily computationally heavy.

Feature importance and observations

The logistic regression model places importance in coefficient magnitudes. The linear SVM uses weight vectors. Finally, the data set's structure itself is important to analyze after being normalized. Over the linear models the features that were the most important were the element mass fractions in the mineral groups.

The less important features were the minor trace elements because they had little variation and the features layed on the same line. This makes sense because minerals in real life have very distinct chemical compositions, so features that identify these variations are the strongest factors.

Model difficulty to interpret

The linear models: logistic and linear SVM were easier to interpret because their coefficients are correlated to feature impacts on class decisions and the separation between data points in the feature set. This helps to realize which features are the most important.

The hardest to interpret was the RBF SVM this is because the decisions are made on nonlinear combinations which do not lead to a single set of feature weights to interpret.

Building Binary Classifiers

In []: %% Task 5: Binary Logistic Regression – Carbonate vs Silicate

```
clear; clc; close all;

% Load normalized training data + family labels
load("mineral_data.mat", "XTrainNorm", "yTrainFamily");

% Choose two families
famA = "Carbonate";
famB = "Silicate";

% Keep only those samples
mask = (yTrainFamily == famA) | (yTrainFamily == famB);
Xbin = XTrainNorm(mask, :);
yFam = yTrainFamily(mask);

% Map labels to 0/1 (0 = famA, 1 = famB)
y = double(yFam == famB);

% (Optional) shuffle the data
N = size(Xbin,1);
idx = randperm(N);
Xbin = Xbin(idx,:);
y = y(idx);

[N, d] = size(Xbin);
fprintf("Binary dataset size: N = %d, d = %d\n", N, d);
fprintf("Class balance: %d %s, %d %s\n", ...
        sum(y==0), famA, sum(y==1), famB);

% Common hyper-parameters
tol = 1e-3;
maxIter = 100000
```

Binary dataset size: N = 1484, d = 1372
 Class balance: 498 Carbonate, 986 Silicate

maxIter =

100000

Experiment 1

```

In [ ]: %% Experiment 1: Plain logistic regression + full-batch GD

% learning rate
etaGD = 0.1;
w1 = zeros(d,1);
b1 = 0;

loss_hist_1 = zeros(maxIter,1);

for t = 1:maxIter

    z = Xbin*w1 + b1;
    p = 1 ./ (1 + exp(-z));

    % small constant limiter so it does not accidentally hit log 0
    eps_val = 1e-12;
    L = -mean( y.*log(p+eps_val) + (1-y).*log(1-p+eps_val) );
    loss_hist_1(t) = L;

    if L < tol
        break;
    end

    % gradients calculated on the loss plane
    diff = p - y;
    grad_w = (Xbin' * diff) / N;
    grad_b = sum(diff) / N;

    % updating the gradients
    w1 = w1 - etaGD * grad_w;
    b1 = b1 - etaGD * grad_b;
end

iters_plain_GD = t;
loss_hist_1 = loss_hist_1(1:t);

fprintf("Exp 1 (logreg + GD): iters = %d, final loss = %.5e\n", ...
        iters_plain_GD, L);

```

Exp 1 (logreg + GD): iters = 27086, final loss = 9.99970e-04

```
In [ ]: %% Experiment 2: L2-regularized logistic regression + GD

% learning rate fixed the same as the first regression to make it comparable
etaGD2 = 0.1;
lambda2 = 1e-2;

w2 = zeros(d,1);
b2 = 0;

loss_hist_2 = zeros(maxIter,1);

for t = 1:maxIter
    z = Xbin*w2 + b2;
    p = 1 ./ (1 + exp(-z));

    eps_val = 1e-12;

    % logistic included with penalty term

    baseL = -mean( y.*log(p+eps_val) + (1-y).*log(1-p+eps_val) );
    L = baseL + 0.5*lambda2*norm(w2)^2;
    loss_hist_2(t) = L;

    if L < tol
        break;
    end

    diff = p - y;
    grad_w = (Xbin' * diff) / N + lambda2 * w2;
    grad_b = sum(diff) / N;

    w2 = w2 - etaGD2 * grad_w;
    b2 = b2 - etaGD2 * grad_b;
end

iters_reg_GD = t;
loss_hist_2 = loss_hist_2(1:t);

fprintf(" Experiment 2 (L2 logreg + GD): iters = %d, final loss = %.5e\n", ...
        iters_reg_GD, L);
```

Experiment 2 (L2 logreg + GD): iters = 100000, final loss = 9.98450e-02

```
In [ ]: %% Experiment 3: Plain logistic regression + SGD

etaSGD = 0.01;
w3 = zeros(d,1);
b3 = 0;

maxIter_SGD = 200000;
loss_hist_3 = [];
check_every = N;

for t = 1:maxIter_SGD
    % picking a random index
    i = randi(N);
    xi = Xbin(i,:);
    yi = y(i);

    zi = xi' * w3 + b3;
    pi = 1 / (1 + exp(-zi));

    % gradient calculations

    diff = pi - yi;
    grad_w_i = diff * xi;
    grad_b_i = diff;

    % updating the SGD
    w3 = w3 - etaSGD * grad_w_i;
    b3 = b3 - etaSGD * grad_b_i;

    % Computing the loss to make sure it is converging

    if mod(t, check_every) == 0
        z = Xbin*w3 + b3;
        p = 1 ./ (1 + exp(-z));
        eps_val = 1e-12;
        L = -mean( y.*log(p+eps_val) + (1-y).*log(1-p+eps_val) );
        loss_hist_3(end+1) = L; %#ok<SAGROW>
        if L < tol
```

```

        break;
    end
end
end

% number of steps

iters_plain_SGD = t;
fprintf("Experiment 3 (plain logreg + SGD): SGD steps = %d, last checked loss = %.3e\n", ...
        iters_plain_SGD, L);

```

Experiment 3 (plain logreg + SGD): SGD steps = 200000, last checked loss = 1.276e-03

In []: %% Experiment 4: L2-regularized logistic regression + SGD

```

etaSGD2 = 0.01;
lambda4 = 1e-2;

w4 = zeros(d,1);
b4 = 0;

maxIter_SGD2 = 200000;
loss_hist_4 = [];
check_every = N;

for t = 1:maxIter_SGD2
    i = randi(N);
    xi = Xbin(i,:);
    yi = y(i);

    zi = xi' * w4 + b4;
    pi = 1 / (1 + exp(-zi));

    diff = pi - yi;

    % gradient + loss term

    grad_w_i = diff * xi + lambda4 * w4;
    grad_b_i = diff;

    w4 = w4 - etaSGD2 * grad_w_i;
    b4 = b4 - etaSGD2 * grad_b_i;

```

```

    if mod(t, check_every) == 0
        z = Xbin*w4 + b4;
        p = 1 ./ (1 + exp(-z));
        eps_val = 1e-12;
        baseL = -mean( y.*log(p+eps_val) + (1-y).*log(1-p+eps_val) );
        L = baseL + 0.5*lambda4*norm(w4)^2;
        loss_hist_4(end+1) = L; %#ok<SAGROW>
        if L < tol
            break;
        end
    end
end

iters_reg_SGD = t;
fprintf("Experiment 4 (L2 logreg + SGD): SGD steps = %d, last checked loss = %.3e\n", ...
        iters_reg_SGD, L);

```

Experiment 4 (L2 logreg + SGD): SGD steps = 200000, last checked loss = 1.002e-01

In []: *%% Summary of all the gradient tasks*

```

% number of iterations budgeted for
fprintf("\n--- Iterations to reach tolerance (or until stop) ---\n");

fprintf("Plain logreg + GD:    %d full-batch iterations \n", iters_plain_GD);
fprintf("L2 logreg + GD:      %d full-batch iterations \n", iters_reg_GD);
fprintf("Plain logreg + SGD:    %d single-sample steps\n", iters_plain_SGD);
fprintf("L2 logreg + SGD:      %d single-sample steps\n", iters_reg_SGD);

% Computing for gradients

grads_plain_GD = iters_plain_GD * N;
grads_reg_GD   = iters_reg_GD * N;
grads_plain_SGD = iters_plain_SGD;
grads_reg_SGD  = iters_reg_SGD;

fprintf("\n--- Approx gradient computations ---\n");
fprintf("Plain logreg + GD:    %d gradients\n", grads_plain_GD);
fprintf("L2 logreg + GD:      %d gradients\n", grads_reg_GD);
fprintf("Plain logreg + SGD:    %d gradients\n", grads_plain_SGD);
fprintf("L2 logreg + SGD:      %d gradients\n", grads_reg_SGD);

```



```
--- Iterations to reach tolerance (or until stop) ---  
Plain logreg + GD: 20000 full-batch iterations  
L2 logreg + GD: 20000 full-batch iterations  
Plain logreg + SGD: 200000 single-sample steps  
L2 logreg + SGD: 200000 single-sample steps  
  
--- Approx gradient computations ---  
Plain logreg + GD: 29680000 gradients  
L2 logreg + GD: 29680000 gradients  
Plain logreg + SGD: 200000 gradients  
L2 logreg + SGD: 200000 gradients
```

The plain logistic regression model converged to its target loss much faster than the L2 regularized model. Both the GD and SGD had lower final loss for the unregularized model (nearly 0.001) when compared to the L2 regularized version (0.1).

Theoretically this makes sense because regularizing the data changes the landscape. By adding a bias term that restricts the objectives from growing unrestrictedly the minimum loss becomes higher. In conclusion, the L2 model because of its bias term constrains the optimal point making it take longer to converge.

Iterations to reach Tolerance

The GD took fewer iterations because it uses a fuller gradient when compared to the SGD which uses sample gradients. The GD took nearly 20,000 iterations in comparison to the SGD which took nearly 200,000 iterations. Theoretically this makes sense because the SGD uses a noisy approach and takes longer to converge.

However, one key difference is although the SGD took more iterations it performed less gradient computations. The GD computed nearly 3 million gradient calculations and the SGD computed only 200 thousand. Even though the GD converged in a smaller iterative budget it was more computationally expensive per iteration because it uses the entire dataset to compute gradients.

When considering gradient computations the SGD is more efficient because it required fewer computations (nearly 15 times less) even though it uses a noisy approach. If the dataset was expanded to the full one offered on the NASA webpage it would be helpful to use the SGD approach.

Additionally the results spoken about were when the maxIter was restricting to 20,000 for runs on a weaker computer to make comparisons between all the models. I was messing around and increased the budget to 100,000. However, the methods that reach their maximum iteration are expected especially for the L2 approach. This is because the L2 has a higher loss because of

the regularization penalty. The GD implementation slows when reaching the optimal value whereas the SGD bounces around it. Therefore fixing the budget does not directly correlate to reaching the loss threshold.